🎯 NewsLens-AI: Complete Interview Playbook & Project Walkthrough

This guide gives you **everything you need to explain NewsLens-AI in a technical interview—** from a 30-second elevator pitch to a 5-minute architectural deep dive, a metrics cheat sheet, and **20 high-probability interview questions with winning answers** grounded directly in your codebase.

📊 1. "Brag Sheet" — Key Metrics to Memorize

> IMPORTANT
>
> Interviewers love **concrete before/after engineering numbers**. Drop these naturally when explaining your architectural decisions:

Metric / Area	Before Optimization	After Optimization	How You Achieved It
24-Page Broadsheet Ingestion	7–11+ mins (`440s–660s`)	**45s–65s (~10x faster)**	In-memory PyMuPDF PDF slicing (`<30ms`), 4-worker concurrent Docling OCR, 8-worker MinIO/GCS raster uploads, in-memory entity cache, and whole-issue bulk Qdrant vector upsert (,).
Query Time-to-First-Token (TTFT)	25s–45s	**1.8s–2.4s (~10x faster)**	Direct Vertex AI routing (`gemini-2.5-flash`), calibrated `thinking_budget: 0` on structured JSON planning/CRAG/synthesis, and CRAG Dual-Signal Fast-Floor (<5ms) (,).
Full Streamed Answer Latency	35s–50s	**3.5s–5.0s**	Parallel tool execution via `asyncio.gather` () + SSE token streaming ().
Cross-Encoder Reranking	2.2s–2.8s	**~450ms (5x faster)**	Process-wide model caching (`_SHARED_CROSS_ENCODERS`) + adaptive candidate capping ($K = 8 \ldots 16$K = 8 ... 16) on CPU (,).
DB Queries per Issue Ingest	~2,200 roundtrips	**3 bulk SQL queries**	Process-level in-memory Entity & Topic lookup caches + bulk SQL persistence ().
Cloud Run Compute & RAM Cost	2.4GB+ PyTorch RAM, 24/7 CPU	**~350MB RAM, 85–90% lower cost**	Offloaded local PyTorch weights to `gemini-embedding-001` (768d MRL) & IBM Docling Cloud SaaS; enabled request-based `--cpu-throttling` on API container ().
System Scale & Reliability	Prototype	**574 passing tests, 17 SQL tables, 8 agentic tools**	Full CI/CD with keyless Workload Identity Federation (OIDC) to GCP (`asia-south1`).

🎙️ 2. How to Introduce the Project (Word-for-Word Scripts)

Choose the version below based on how much time the interviewer gives you (or start with the 2-minute version and let them ask follow-ups).

⚡ Version A: The 30-Second Elevator Pitch

(Use this during "Tell me about yourself" or a quick resume scan)

> "I built **NewsLens-AI**, an enterprise-grade **Agentic RAG and Multimodal Broadsheet Intelligence Platform** designed to turn complex, multi-column scanned newspaper archives into searchable, citation-backed intelligence.
>
> Standard RAG fails on newspapers because PDF parsers read straight across 6 to 8 vertical columns, scrambling unrelated stories together, and can't read infographics or answer database-wide statistical questions.
>
> To solve this, I engineered a **2D spatial ingestion pipeline** using **IBM Docling, RapidOCR, and Vision-Language Models** that segments multi-column articles, binds photos to captions via 2D proximity scoring, and ingests a 24-page newspaper in **under 60 seconds.** On the query side, I built a **LangGraph state-machine agent** orchestrating **8 specialized tools**—including hybrid dense+sparse retrieval with Reciprocal Rank Fusion and Cross-Encoder reranking, Corrective RAG (CRAG), and an AST-sandboxed dynamic Python/SQL tool synthesizer—delivering streaming responses with spatial bounding-box citations in **under 2 seconds TTFT.**"

🏗️ 3. Visual Architecture Flow (Mental Model for Whiteboard Rounds)

If asked to draw the system architecture in an interview, draw these **two pipelines** (Ingestion & Agentic Query):

❓ 4. Top 20 Interview Questions & Winning Answers

Category A: Project Motivation & Core Design Decisions

Q1: Why couldn't you just use standard LangChain PDF loaders and naive chunking for newspapers?

- **What they're testing:** First-principles understanding of document intelligence vs. tutorial-level RAG.
- **How to answer:**
    1. **Multi-Column Reading Order Corruption:** Standard PDF loaders read left-to-right across the full page width. On an 8-column broadsheet, line 1 of Column 1 gets concatenated with line 1 of Column 2 from a completely different story. I used **IBM Docling (DocLayNet)** and custom **2D spatial column de-bundling** () to reconstruct vertical reading order and attach multi-column headlines/decks to the right body columns.
    2. **Spatial Citation Loss:** Users need to verify where a quote came from on a dense page. We preserve normalized 2D bounding boxes (`x, y, w, h`) from chunking all the way to the React canvas.
    3. **Relational & Negative Queries:** Vector search only finds top-kk similar chunks. It fails on *"How many articles were published on page 1?"* or *"Which stories did The Hindu cover that The Goan missed?"* That's why **MySQL 8** is our System of Record alongside **Qdrant**, backed by `sql_analytics` and `coverage_analysis` .

Q2: Why did you choose MySQL 8 + Qdrant instead of PostgreSQL with `pgvector` ?

- **What they're testing:** Database trade-offs and architectural justification.
- **How to answer:**
    - **Separation of Concerns & Dedicated Rust Vector Engine: Qdrant** (written in Rust) gives us sub-15ms HNSW vector search with rich payload filtering (`newspaper_name` , `issue_date` , `section` , `has_visual_data`) and allowed us to build a **strict dual-collection architecture** (`article_chunks` at 1024d for local BGE-M3 and `article_chunks_v2` at 768d for Cloud Gemini MRL) without running heavy `ALTER TABLE` vector migrations on relational tables ().
    - **MySQL 8 FULLTEXT + Relational Integrity:** MySQL 8 handles our 17 normalized relational tables (`newspapers` , `issues` , `pages` , `articles` , `article_chunks` , `photos` , `entities` , `article_entities` , etc.) and native `MATCH(...) AGAINST(... IN NATURAL LANGUAGE MODE)` BM25 sparse search used in our Hybrid Search pipeline ().

Category B: Document Ingestion, OCR & Multimodal Vision

Q3: How did you speed up 24-page newspaper ingestion from ~10 minutes down to ~50 seconds without losing OCR quality?

- **What they're testing:** Systems performance engineering, concurrency, and I/O bottleneck elimination.
- **How to answer:** Walk through the **5 bottlenecks** you eliminated in :
    1. **Upfront In-Memory PDF Slicing (`<30ms`):** Instead of opening the 24-page PDF from disk 24 separate times, we slice single-page byte streams in memory with PyMuPDF and run **4 concurrent Docling OCR workers** (`asyncio.Semaphore(4)`).
    2. **8x Concurrent Page Rasterization & Upload:** Rendered page PNGs in memory and uploaded to MinIO/GCS with `asyncio.Semaphore(8)` plus a single bulk SQL update.
    3. **Calibrated VLM Token Budgets & Sub-Batching:** For visual extraction, we partitioned page assets into sub-batches of ≤ 6≤ 6 items with an 8,192-token output runway and dynamic thinking budgets (`512` tokens for tables/infographics, `128` for photos) across 4 workers so one large table never truncates the JSON response.
    4. **In-Memory Entity/Topic Caching:** Cut **~2,200 individual SQL roundtrips** per issue down to **3 bulk SQL queries**.
    5. **Whole-Issue Bulk Vector Indexing:** Instead of embedding and upserting article-by-article, aggregates all ~250 chunks across the issue, embeds them in 2 batch API calls, and performs **1 bulk Qdrant upsert** and **1 SQL commit**.

Q4: What happens when a scanned PDF has corrupted font encodings (`\ufffd` replacement characters) or when the Vision-Language Model fails on a dense financial table?

- **What they're testing:** Edge-case resilience and deterministic fallbacks in production AI pipelines.
- **How to answer:**
    - **Corrupted CMap Recovery ():** Many Indian and historical newspaper PDFs have broken `ToUnicode` font maps where digital text extraction yields gibberish or `\ufffd` characters. Our pipeline measures character corruption ratios per page; when corruption is detected, it automatically discards the embedded PDF text stream and escalates to pure image-based OCR (`GoogleCloudVisionOCR` / `RapidOCR`) with `LayoutAnalyzer` .
    - **Deterministic Spatial OCR Matrix Fallback ():** When local or cloud VLMs return empty output or time out on high-density financial tables, our deterministic spatial OCR matrix engine clusters OCR token bounding boxes by YY -row alignment and XX -

column boundaries to reconstruct structured Markdown tables with confidence ≥ 0.85 ≥ 0.85.

- **Post-OCR Semantic Reconstruction ():** At synthesis time, explicit prompt rules authorize the LLM to phonetically and semantically repair minor OCR typos (*"Reconstruction is not hallucination"*) so raw scan artifacts never appear in user answers.

Q5: How does your system associate photos and infographics with the right article and caption on a cluttered newspaper page?

- **What they're testing:** Spatial geometry algorithms and multimodal indexing.
- **How to answer:**
  - In and , we compute a **2D spatial proximity score** between every extracted visual bounding box and nearby text blocks, weighting vertical adjacency (directly below/above) and horizontal column overlap to bind captions to photos.
  - For composite photo galleries where heuristic bounding boxes merge multiple portraits into one block, we use **VLM spatial grounding** (`detect_subphotos_via_vlm_grounding`) to detect and crop discrete sub-photos.
  - Furthermore, if a single page has a bad scan or updated model config, users can trigger **atomic single-page re-ingestion** (`POST /api/issues/{issue_id}/pages/{page_number}/reingest` in), which purges only that page's articles and Qdrant vectors and re-processes it without touching the other 23 pages.

Category C: Hybrid Search, Embeddings & Vector Architecture

Q6: Explain your Two-Stage Hybrid Retrieval pipeline. Why use both Reciprocal Rank Fusion (RRF) and a Cross-Encoder?

- **What they're testing:** Information Retrieval (IR) depth—bi-encoders vs. lexical search vs. cross-encoders.
- **How to answer:**
  - **Stage 1 — Hybrid Candidate Generation ($N = 75$N = 75):**
    - **Dense Track (Qdrant):** Captures semantic meaning and synonyms (e.g., matching *"monetary policy tightening"* to *"RBI hikes repo rate"*).
    - **Sparse Track (MySQL FULLTEXT BM25):** Captures exact lexical matches (proper nouns, section numbers, statutory acts, exact quotes) where dense embeddings often blur details.

        - **Reciprocal Rank Fusion (RRF, $k = 60$k = 60):** Since cosine similarity scores ($0 \dots 10 \dots 1$) and MySQL BM25 scores (unbounded) are on incompatible scales, we fuse their rankings via: $\text{RRF_Score}(d) = \sum_{m \in \{\text{dense}, \text{sparse}\}} \frac{1}{60 + \text{rank}_m(d)}$ RRF_Score(d) = ∑m∈{dense,sparse} 60+rankm(d)1
    - **Stage 2 — Neural Cross-Encoder Reranking ():**
        - Bi-encoders compress query and document into single vectors independently, losing fine-grained token-to-token cross-attention.
        - A Cross-Encoder (`cross-encoder/ms-marco-MiniLM-L-6-v2`) jointly encodes `(query, document_chunk)` pairs through full self-attention for high precision.
        - **Optimization:** Running a Cross-Encoder on all 75 candidates took ~2.8s on CPU. I implemented **adaptive candidate pool capping**: $\text{max_rerank_candidates} = \min(\text{len(results)}, \min(16, \max(8, \text{top_k} \times 2)))$ max_rerank_candidates = min(len(results), min(16, max(8, top_k × 2))) and cached the model at the process level (`_SHARED_CROSS_ENCODERS`), cutting reranking latency to **~450ms** with zero loss in Top-5 precision.

Q7: How did you handle switching between local embeddings (`BAAI/bge-m3` , 1024d) and cloud embeddings (`gemini-embedding-001` , 768d) without breaking Qdrant?

- **What they're testing:** Vector database schema design, Matryoshka embeddings, and zero-downtime migrations.
- **How to answer:**
    - A Qdrant collection has a fixed vector dimension. Local `BAAI/bge-m3` outputs **1024-dimensional** vectors, whereas in production Cloud Run we wanted **zero container RAM overhead** (saving 2.4GB of PyTorch weights) by using Google's `gemini-embedding-001` truncated to **768 dimensions via Matryoshka Representation Learning (MRL)** with asymmetric task types (`RETRIEVAL_DOCUMENT` at ingest, `RETRIEVAL_QUERY` at search).
    - I architected **Strict Dual-Collection Auto-Routing** in : `get_collection_name(dim)` inspects the vector length at runtime and routes 1024d vectors to `article_chunks` and 768d vectors to `article_chunks_v2` .
    - When an issue is deleted or re-ingested, `delete_by_filter()` purges matching payloads across **both** collections so no orphan vectors remain, and backfills historical MySQL chunks into `article_chunks_v2` in batches.

Category D: Agentic RAG, LangGraph, CRAG & Dynamic Tool Synthesis

Q8: Why did you use a Single-Turn Cognitive Planner with Parallel Tool Execution instead of a standard ReAct loop?

- **What they're testing:** Agentic latency, reliability, and orchestration design.
- **How to answer:**
  - Traditional ReAct loops (`Thought -> Action -> Observation -> Thought...`) make sequential LLM calls after every single tool execution. If an agent calls 3 tools sequentially, you pay 3–4x the LLM latency (easily 20–30 seconds) and risk infinite tool-calling loops.
  - Instead, in and , we first inject live archive metadata (`get_archive_metadata`: available newspapers, date ranges, section taxonomy) into the **Cognitive Planner LLM**.
  - In a **single structured Pydantic call**, the planner outputs:
    1. A list of 1 to 3 objects, which executes **concurrently via `asyncio.gather`**.
    2. A tailored (`SectionSpec` defining presentation formats, markdown tables, metric cards, and word budgets) so presentation planning happens upfront without extra LLM hops.

Q9: How does your Corrective RAG (CRAG) loop work, and how do you keep it from adding 4–5 seconds of latency to every query?

- **What they're testing:** Self-reflective RAG (CRAG) and latency optimization.
- **How to answer:**
  - In , running an LLM-as-a-Judge on every single query would add ~4 seconds of latency even when retrieval clearly succeeded.
  - To solve this, I engineered a **Two-Tier CRAG Evaluator**:
    1. **Tier 1 — Dual-Signal Fast-Floor Gate (`<5ms`):** Deterministically inspects the retrieved evidence. If we have ≥ 1≥ 1 relevant article, ≥ 100≥ 100 words, AND strong neural retrieval confidence (`rerank_score >= 0.30` or `rrf_score >= 0.015`), it immediately approves the evidence in **under 5 milliseconds** and skips the LLM Judge.
    2. **Tier 2 — Reflexive LLM Judge (`evaluate_evidence_async`):** Only triggered when evidence is sparse, low-scoring, or ambiguous. It outputs a typed `EvaluationVerdict` (`quality_score`, `gap_diagnosis`, `recommended_action`, `corrective_hints`).
  - **Closed-Loop Recovery (1-Cycle Ceiling):** If the Judge diagnoses a gap, LangGraph routes to `execute_adaptive_replan` (which widens date bounds, relaxes category

filters, expands `top_k`, or dispatches `dynamic_analysis`) under a strict **1-cycle ceiling** with an anti-repetition guard so it never loops infinitely.

- **Legitimate Absence Discrimination:** If a user asks about a topic that genuinely wasn't published in that day's newspaper, CRAG distinguishes **legitimate zero coverage** from retrieval failure so the synthesizer reports verified absence instead of hallucinating.

Q10: What happens if a user asks an analytical question that your 7 pre-compiled `sql_analytics` routines cannot answer (e.g., *"What is the correlation between article word count and page number across all August issues?"*)?

- **What they're testing:** Dynamic code generation, AI safety, and sandboxing.
- **How to answer:**
  - That triggers our 3rd tool: **`dynamic_analysis`**, powered by , , and .
  - **Step 1 — Code Synthesis (`ToolMaker`):** The LLM writes a custom Python + SQL analytical function tailored to the live MySQL schema.
  - **Step 2 — 5-Dimension Audit (`ToolCritic`):** Before execution, `ToolCritic` evaluates the generated code across 5 quantitative metrics:
    1. **SASC** (Schema & SQL Correctness)
    2. **SRF** (Security & Resource Footprint)
    3. **REH** (Resilience & Error Handling)
    4. **DSF** (Data Structure Fidelity)
    5. **RPS** (Relevance to Problem Statement) If it fails the threshold, `ToolMaker` refines the code with the critic's feedback.
  - **Step 3 — Multi-Layer Isolated Execution (`SandboxedExecutor`):**
    1. **Static AST Scanner (`ASTSafetyScanner`):** Parses the Python Abstract Syntax Tree to block forbidden imports (`os`, `sys`, `subprocess`, `socket`, `shutil`), dunder attribute access (`__subclasses__`), and destructive SQL (`DROP`, `DELETE`, `UPDATE`, `INSERT`, `ALTER`).
    2. **Subprocess Isolation:** Executes in a separate OS subprocess with `RLIMIT_AS` capped at **512MB RAM**, a strict **15-second timeout**, and a **read-only MySQL transaction** that always issues `ROLLBACK` on exit.

Q11: How do you prevent hallucinations in the final synthesized response?

- **What they're testing:** Trustworthiness, grounding, and provenance in enterprise GenAI.
- **How to answer:** Mention your **6-Layer Hallucination Defense** (documented in):

1. **Live Archive Grounding at Planning Time:** The Planner knows exact available newspapers and dates (`get_archive_metadata`), preventing queries against non-existent dates.
2. **Cross-Date & Cross-Turn Isolation Shield ():** Purges stale `article_id` , `photo_id` , and `target_newspapers` whenever the user switches dates, publications, or topics across chat turns.
3. **Single-Article Context Isolation ():** When explaining a specific article, filters out adjacent advertisements or unrelated stories from the same page so they don't bleed into the summary.
4. **CRAG Evidence Verification & 3-Tier Negative Coverage Engine:** Explicitly audits which newspapers had `0` matching articles so the LLM states *"Zero coverage in The Goan"* instead of fabricating a viewpoint.
5. **Strict Inline Citation Contract:** Every factual claim must carry `[Newspaper, YYYY-MM-DD, Page N, "Headline"]` , parsed by `parse_inline_citation()` and mapped to database article IDs and 2D bounding boxes.
6. **Post-Synthesis Answer Verifier ():** Audits generated responses against retrieved evidence chunks before completion.

Category E: Performance Optimization, Cloud Architecture & DevOps

Q12: How did you reduce query Time-to-First-Token (TTFT) from 25–45 seconds down to 1.8–2.4 seconds?

- **What they're testing:** LLM inference latency profiling and reasoning model tuning.
- **How to answer:**
  1. **Eliminating Hidden Thinking Token Latency (`thinking_budget: 0`):** Reasoning-capable models like `gemini-2.5-flash` generate internal chain-of-thought tokens by default before emitting JSON or stream tokens, adding 15–20 seconds of silent latency across the Planner, CRAG Judge, and Synthesizer. Since our Planner already uses explicit structured Pydantic Chain-of-Thought fields (`reasoning` , `query_archetype` , `AnswerBlueprint`), I set `thinking_budget: 0` on structured planning, CRAG evaluation, and streaming synthesis, and filtered `part.get("thought")` from SSE streams.
  2. **Direct Vertex AI Publisher Routing:** Eliminated legacy model alias 404 retries by routing directly to `aiplatform.googleapis.com` with `gemini-2.5-flash` .
  3. **CRAG Fast-Floor (`<5ms`):** Bypassed the 4s LLM Judge when retrieval scores (`rerank_score >= 0.30`) and word counts (≥ 100≥ 100 words) are already high.

4. **Adaptive Cross-Encoder Capping:** Cut CPU reranking from 2.2s to ~450ms.

Q13: How do you handle token limits when switching between models (e.g., local 8B Ollama models vs. reasoning models like DeepSeek-R1 or Gemini 2.5)?

- **What they're testing:** Multi-provider abstraction and token budgeting.
- **How to answer:**
  - I built `resolve_dynamic_token_budget()` in backed by .
  - Instead of hardcoded `max_tokens`, the system inspects `ProviderCapability` (`max_output_tokens`: 4,096 or 8,192) and auto-detects reasoning models (`is_reasoning_model: true` for DeepSeek-R1, QwQ, Nemotron, Gemini thinking modes).
  - For reasoning models, it automatically adds `reasoning_headroom = 2048` using the formula: $\text{effective_max_tokens} = \max(1024, \text{target_words} \times 4) + \text{reasoning_headroom}$ effective_max_tokens = max(1024, target_words × 4) + reasoning_headroom so internal `<think>` tokens never starve or truncate the final user-facing response.

Q14: Walk me through your Google Cloud Run deployment and how you cut cloud costs by 85–90%.

- **What they're testing:** Cloud architecture, FinOps, and serverless constraints.
- **How to answer:**
  - We deploy 3 containerized services + 1 migration job in `asia-south1` via keyless GitHub Actions (Workload Identity Federation OIDC):
    1. **`newslens-frontend`:** Nginx 1.27 Alpine container serving the React SPA and proxying `/api/*` with `proxy_buffering off` so Server-Sent Events (SSE) stream instantaneously.
    2. **`newslens-backend`:** FastAPI with `--cpu-throttling` (request-based billing). CPU is only billed while an HTTP request or SSE stream is active; idle time costs ₹0 (saving ~85% compute cost).
    3. **`newslens-worker`:** Cloud Run requires every container to listen on `$PORT` and pass HTTP health checks, and throttled CPU would kill background Celery workers mid-OCR. So I built , which spins up a lightweight background HTTP health server on `$PORT` while running Celery in the foreground with `--no-cpu-throttling` and `--min-instances=1` connected to Upstash Redis TLS.
  - **Vertex AI ADC Credit Routing:** By authenticating via the Cloud Run Service Account's Application Default Credentials (ADC) directly to Vertex AI endpoints rather than AI

Studio API keys, 100% of LLM and embedding spend is covered by GCP Promotional Credits.

Category F: Deep Technical & Behavioral ("Hardest Bug") Questions

Q15: How does your Deterministic Cross-Newspaper Differential Coverage Engine work?

- **What they're testing:** Algorithmic problem solving beyond LLM prompting.
- **How to answer:**
  - When a user asks *"What news was in The Goan on Aug 14 that wasn't in The Hindu?"*, an LLM with top-10 vector search cannot know the full universe of articles in both papers.
  - Our `sql_analytics` routine `get_newspaper_coverage_difference` () fetches the complete article rosters for Newspaper A and Newspaper B on that date from MySQL.
  - It then runs **normalized headline & entity token overlap scoring** (stripping stopwords and comparing significant lexical tokens and named entities) to deterministically partition articles into **Exclusive to Newspaper A**, **Exclusive to Newspaper B**, and **Shared Wire/Syndicated Stories**, returning exact page folios and sections.

Q16: How does the Interactive Visual Asset Inspector (`inspect_visual_asset`) work when a user clicks "Ask Agent About This Infographic" on the frontend?

- **What they're testing:** Full-stack multimodal integration (React UI → FastAPI → LangGraph → MinIO/GCS → VLM).
- **How to answer:**
  - On the React 18 Broadsheet Reader, every detected photo, chart, and table has an interactive bounding-box overlay. Clicking *"Ask Agent About This Infographic"* attaches an `attachedAsset` pill (`photo_id` , `issue_id` , `page_number`) to the chat request.
  - In the backend, executes a **5-Tier Strategy Cascade**:
    - **Strategy A:** Direct `photo_id` lookup + automatic retrieval of companion sub-charts from the same parent article/page.
    - **Strategy B:** Headline match lookup.
    - **Strategy C:** `article_id` lookup + companion visual assets.
    - **Strategy D:** Multi-criteria relational DB search (newspaper + date + page + section).
    - **Strategy E:** Scoped caption and VLM transcript semantic search.

- If the stored visual description is only a lightweight placeholder, `VisualInspectionEngine` triggers **lazy on-demand VLM extraction**, streaming the high-res image crop from MinIO/GCS (`bucket_pages`) to `gemini-2.5-flash` / `Qwen3-VL` to extract full Markdown tables on the fly and cache them back into MySQL.

Q17: What was the hardest bug you encountered while building NewsLens-AI, and how did you fix it?

- **What they're testing:** Debugging rigor, root-cause analysis, and production engineering maturity.
- **Pick one of these 3 real bugs from your engineering log ():**
  - **Story 1 — Cross-Turn Date & Asset Contamination in Multi-Turn Chat:**
    - *Symptom:* A user would inspect an infographic from *Aug 1* in Turn 1, and then ask *"What were the top headlines on Aug 14?"* in Turn 2. The agent kept pulling *Aug 1* context or binding the stale `photo_id` from Turn 1.
    - *Root Cause:* The conversational condenser and graph state were carrying forward `attached_asset` IDs and historical date filters even when the new query explicitly specified a different date.
    - *Fix:* Engineered the **Cross-Date Context Isolation & Anti-Leakage Shield** across , , and : explicit query dates strictly override prior history, and switching dates, newspapers, or topics triggers 3 parameter eviction guardrails that purge stale `article_id` , `photo_id` , and `headline` bindings.
  - **Story 2 — Reasoning Model Token Starvation:**
    - *Symptom:* When testing reasoning models (Gemini 2.5 Flash with thinking enabled / DeepSeek-R1) on complex cross-newspaper comparisons, responses were getting cut off mid-sentence or returning empty strings despite setting `max_tokens=2048` .
    - *Root Cause:* Reasoning models consume output token budget for internal `<think>` chains before emitting visible text. A 1,800-token internal thought left only 200 tokens for the actual executive brief.
    - *Fix:* Built `resolve_dynamic_token_budget()` in to add 2,048 tokens of `reasoning_headroom` for reasoning models, and set `thinking_budget: 0` on latency-critical JSON planning and streaming synthesis calls where structured blueprints already provide the reasoning scaffold.
  - **Story 3 — Apple Silicon MPS Metal Shader Lag / CPU Reranker Spikes:**
    - *Symptom:* Cross-Encoder reranking was taking 30–40s on initial queries and 2.5s+ on subsequent queries.

        - *Root Cause:* Re-instantiating `CrossEncoder` reloaded weights from disk on every request, and PyTorch MPS graph compilation caused stalls on macOS while 75 candidates saturated 1 vCPU in Cloud Run.
        - *Fix:* Added process-wide singleton caching (`_SHARED_CROSS_ENCODERS`), forced optimized CPU execution for the lightweight MiniLM cross-encoder, and capped reranked candidates adaptively to $K = 8 \ldots 16$K = 8 ... 16, dropping latency to `~450ms`.

Q18: How do you evaluate the quality of your RAG pipeline quantitatively?

- **What they're testing:** ML evaluation rigor (`backend/app/evaluation/`).
- **How to answer:**
    - We have an evaluation subsystem in that tracks both **Retrieval IR metrics** (Precision@KK, Recall@KK, Mean Reciprocal Rank / MRR, and NDCG across hybrid search vs. dense-only) and **Generation faithfulness metrics** (citation accuracy, groundedness via , and CRAG quality scores).
    - Additionally, our test suite () runs **574 unit and integration tests** validating parser geometry, SQL analytics, AST sandbox security, cross-date isolation, and agentic tool routing on every GitHub Actions push.

Q19: How does the Cross-Newspaper Narrative Trajectory / Story Timeline (`timeline_builder`) prevent grouping unrelated articles together?

- **What they're testing:** Temporal clustering and anti-hallucination gates.
- **How to answer:**
    - In , reconstructing how a story evolved over weeks across multiple newspapers requires more than simple vector similarity (which might group two unrelated court cases just because both mention "Supreme Court").
    - We enforce **4-tier anti-hallucination gates** combining semantic similarity thresholds, shared named entity verification, temporal ordering across issue dates, and LLM narrative phase classification (`Breaking`, `Development`, `Financial Impact`, `Regulatory/Outcome`), backed by Redis caching for instant repeat lookups.

Q20: If you had another month to work on NewsLens-AI, what would you build or improve next?

- **What they're testing:** Product vision, scalability awareness, and engineering maturity.
- **How to answer:**
    1. **Cross-Lingual Vernacular Broadsheet Alignment:** Expand OCR + `BAAI/bge-m3` / Gemini multilingual embeddings so a single English query seamlessly compares

coverage across English (*The Hindu*), Hindi (*Dainik Jagran*), and Marathi/Konkani regional dailies in the same comparison matrix.

2. **Automated Daily E-Paper Ingestion Cron & Alerting:** Add scheduled ingestion pipelines with real-time webhook/email alerts when specific tracked entities (e.g., a company, regulatory body, or competitor) appear in morning print editions, including automated sentiment and prominence trend tracking.
3. **Distributed Ray / Cloud Run Jobs for Massive Historical Backfills:** For digitizing 50-year newspaper archives (100,000+ issues), shard ingestion across ephemeral Cloud Run Jobs with batched Vertex AI Batch Prediction APIs to cut backfill costs by another 50%.